

Lekce 13: Praktické použití TypeScriptu s Cypress

Cíle

- Seznámit se s TypeScriptem a jeho výhodami v kontextu testování s Cypress.
- Nastavit TypeScript v projektu Cypress.
- Psát a spouštět Cypress testy s využitím TypeScriptu, včetně anotací typů a rozhraní.
- Naučit se osvědčené postupy pro organizaci TypeScript souborů a zachování typové bezpečnosti v testech.

Rozdělení obsahu

A. Úvod do TypeScriptu

1. Co je TypeScript?

- **Definice:**

TypeScript je staticky typovaný nadmnožina jazyka JavaScript, která je překládána na běžný JavaScript. Přidává volitelné typy, rozhraní a pokročilé nástroje do JavaScriptu.

- **Klíčové vlastnosti:**

- Statická kontrola typů: Odhaluje chyby při kompilaci.
- Lepší podpora v IDE: Rozšířený IntelliSense, automatické doplňování a inline dokumentace.
- Moderní JS funkce: Podpora ES6/ES7 funkcí a novějších, s zpětnou kompatibilitou.

1. Co znamená „nadmnožina JavaScriptu“?

Definice:

- Nadmnožina je jazyk, který zahrnuje všechny funkce jiného jazyka (v tomto případě JavaScriptu) a přidává nad ně další vlastnosti.

Vysvětlení:

- **TypeScript je nadmnožinou JavaScriptu:**

To znamená, že jakýkoli platný JavaScriptový kód je také platný TypeScriptový kód. TypeScript přidává nové vlastnosti (např. statické typy, rozhraní, enumy), které v čistém JavaScriptu nejsou dostupné.

- **Proč je to důležité:**

Vývojáři mohou přecházet na TypeScript postupně přidáváním typových anotací a dalších vlastností do existujícího kódu v JavaScriptu, aniž by museli vše přepisovat od začátku.

2. Co je to kompilátor?

Definice:

- Kompilátor je program, který překládá kód napsaný v jednom programovacím jazyce (zdrojovém) do jiného jazyka (cílového). U TypeScriptu kompilátor (tsc) převádí TS kód na běžný JavaScript.

Vysvětlení:

- **TypeScript kompilátor (tsc):**

Provádí kontrolu typů a převádí rozšířenou syntaxi TypeScriptu na běžný JavaScript, který může běžet v prohlížeči nebo Node.js.

- **Role ve vývoji:**

Kompilátor pomáhá zachytit chyby typů už v době překladu, což zlepšuje kvalitu kódu před jeho spuštěním.

2. Výhody použití TypeScriptu s Cypress:

- **Zlepšená kvalita kódu:**

Kontrola typů pomáhá rychleji nalézt chyby během vývoje.

- **Lepší vývojářský zážitek:**

Bohatý IntelliSense a doplňování v editorech jako VS Code usnadňují a zrychlují psaní testů.

- **Lepší údržba:**

Jasně definice typů a rozhraní usnadňují pochopení kódu a refaktORIZACI.

- **Škálovatelnost:**

Jak testovací sada roste, silné typování pomáhá zachovat konzistenci a snižuje počet chyb při běhu.

B. Nastavení TypeScriptu v Cypress

1. Instalace TypeScriptu a potřebných typů:

- **Instalační příkazy:**

```
npm install --save-dev cypress typescript @types/node
```

- **Proč tyto balíčky?**

- `typescript` : Kompilátor TypeScriptu.
- `@types/node` : Typové definice pro Node.js, potřebné pro Cypress úlohy a Node API.

2. Konfigurace tsconfig.json pro Cypress:

- V kořeni projektu vytvořte soubor `tsconfig.json`, pokud již neexistuje.
- **Příklad tsconfig.json :**

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "allowJs": false,
    "skipLibCheck": true,
    "esModuleInterop": false,
    "allowSyntheticDefaultImports": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "moduleResolution": "node",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "types": ["cypress", "node"]
  },
  "include": [
    "cypress/**/*.ts",
    "cypress/**/*.tsx"
  ],
  "exclude": ["node_modules"]
}
```

- **Vysvětlení:**

- **target** a **module** : Zajišťuje moderní JavaScript výstup.
- **strict** : Povolení striktní kontroly typů.
- **types** : Zahrnuje typové definice Cypress a Node.
- **include** : Ujistí se, že TypeScript kompiluje vaše testovací soubory umístěné ve složce Cypress.

C. Psání Cypress testů v TypeScriptu

1. Převod JavaScript testů na TypeScript:

- Změňte příponu souboru testu z `.js` na `.ts`.
- Používejte anotace typů dle potřeby, např.:

```
describe('User Login', () => {
  it('should log in successfully with valid credentials', () => {
    cy.visit('/login');
    cy.get('[data-testid="login-username-input"]').type('demoUser');
    cy.get('[data-testid="login-password-input"]').type('demoPass');
    cy.get('[data-testid="login-submit-button"]').click();
    cy.get('[data-testid="login-success-message"]').should('be.visible');
  });
});
```

- Všimněte si, že Cypress příkazy stále pracují bez problémů díky definicím v `@types/cypress`.

2. Používání anotací typů a rozhraní:

- **Definice rozhraní:**

Pomocí rozhraní definujte tvar datových objektů očekávaných v testech.

```
interface User {
  username: string;
  email: string;
  password: string;
}
```

- **Použití rozhraní:**

```
const validUser: User = {
  username: 'demoUser',
  email: 'demo@example.com',
  password: 'demoPass'
};
```

```
describe('User Login with TypeScript', () => {
  it('should log in successfully using a User object', () => {
    cy.visit('/login');
    cy.get('[data-testid="login-username-input"]').type(validUser.username);
    cy.get('[data-testid="login-password-input"]').type(validUser.password);
    cy.get('[data-testid="login-submit-button"]').click();
    cy.get('[data-testid="login-success-message"]').should('be.visible');
  });
});
```

Rozdíl mezi typy a rozhraními

Rozhraní:

- **Účel:**

Definují tvar objektu. Používají se hlavně k popisu struktury, kterou má objekt mít.

- **Vlastnosti:**

- Podporují rozšiřování (dědičnost).
- Sloučení deklarací (declaration merging).
- Nejvhodnější pro definování kontraktů v kódu.

- **Příklad:**

```
interface User {  
    username: string;  
    email: string;  
    age?: number; // Nepovinná vlastnost  
}
```

Typy:

- **Účel:**

Typové aliasy mohou definovat typy pro objekty, primitivy, unie, průniky a další.

- **Vlastnosti:**

- Jsou flexibilnější než rozhraní.
- Mohou definovat uniové typy, průnikové typy nebo primitivní typy.
- Nepodporují sloučení deklarací.

- **Příklad:**

```
type User = {  
    username: string;  
    email: string;  
    age?: number;  
};  
  
type Response = User | null; // Příklad uniového typu
```

Kdy použít co:

- Použijte **rozhraní** pokud potřebujete definovat tvar objektu a v budoucnu jej rozšířit.
- Použijte **typy** pokud potřebujete větší flexibilitu (např. uniové typy či komplexní skládání typů).

Co jsou Enumy a jak je použít v Cypress

Definice:

- **Enumy (výčty):**

Enumy umožňují definovat sadu pojmenovaných konstant. Zlepšují čitelnost a údržbu tím, že

dávají popisná jména číselným nebo řetězcovým hodnotám.

Příklad použití:

- V Cypress testu můžete enum použít například pro různé uživatelské role nebo stavy aplikace.

Příklad v TypeScriptu:

```
enum UserRole {  
  Admin = 'admin',  
  User = 'user',  
  Guest = 'guest'  
}  
  
// Použití v testu nebo page objectu  
const currentUserRole: UserRole = UserRole.Admin;  
cy.log(`Current User Role: ${currentUserRole}`);
```

Výhody:

- Zvyšuje čitelnost nahrazením literálů popisnými jmény.
- Pomáhá zajistit, že jsou používány jen povolené hodnoty díky kontrole typů.

Co jsou soubory s příponou .d.ts

Definice:

- **Soubory .d.ts (deklarace typů):**

Jsou to soubory deklarací TypeScriptu, které poskytují typové informace o JS modulech, které nemají své vlastní typy. Pomáhají kompilátoru TS porozumět tvaru knihoven a API.

Použití:

- Typicky se používají ve složce `types` nebo spolu s balíčky, které nemají definice pro TypeScript.
- Deklarují rozhraní, typy a moduly bez implementace.

Příklad:

```
// custom-types.d.ts  
declare module 'my-legacy-library' {  
  export function doSomething(input: string): number;  
}
```

Výhoda:

- Umožňuje integraci běžných JS knihoven do TypeScript projektu s typovou kontrolou a IntelliSense.

3. Využití IntelliSense a kontroly typů:

- Díky TypeScriptu bude vaše IDE (např. VS Code) poskytovat návrhy v reálném čase, automatické doplňování a okamžitou kontrolu chyb.
- Psaní a ladění testů je tak efektivnější a méně chybové.

D. Osvědčené postupy pro TypeScript v Cypress

1. Organizace TypeScript souborů:

- Uchovávejte všechny testovací Cypress soubory s příponou `.ts`.
- Testy organizujte např. do složky `cypress/e2e` a ujistěte se, že `tsconfig.json` obsahuje tyto cesty.
- Znovupoužitelné typy a rozhraní ukládejte do samostatného souboru (např. `cypress/support/types.ts`).

2. Zachování typové bezpečnosti:

- Vždy používejte strict mode (`"strict": true`) ve svém `tsconfig.json`.
- Používejte rozhraní a typové anotace pro testovací data, konfigurační objekty a vlastní příkazy.
- Vyhněte se používání `any`, pokud to není nezbytně nutné.

3. Psát přehledný, modulární kód:

- Rozděľujte logiku testů na menší, spravovatelné funkce nebo vlastní příkazy.
- Používejte strukturu Arrange-Act-Assert (AAA) při psaní případů (uspořádej – proved' – ověř).

4. Použití vlastních příkazů s TypeScriptem:

- Definujte vlastní příkazy v TypeScript souboru (např. `cypress/support/commands.ts`).
- Určete typy pro parametry a návratové hodnoty, aby byla zajištěna lepší kontrola typů a IntelliSense.
- **Příklad vlastního příkazu v TypeScriptu:**

```

export { }
declare global {
  namespace Cypress {
    interface Chainable {
      login(name: string, password: string): Chainable<void>
    }
  }
}

Cypress.Commands.add('login', (username: string, password: string) => {
  cy.get('[data-testid="login-username-input"]').clear().type(username);
  cy.get('[data-testid="login-password-input"]').clear().type(password);
  return cy.get('[data-testid="login-submit-button"]').click();
});

```

Definice návratových typů

- Návratové typy v TypeScriptu explicitně určují typ, který funkce vrátí.

Použití:

- Určení návratových typů pomáhá zachytit chyby, když implementace funkce neodpovídá deklarovanému návratovému typu.

Příklad:

```

function sum(a: number, b: number): number {
  return a + b;
}

const result: number = sum(5, 7);

```

Co znamená Chainable<void> a Chainable<Element>

Definice:

- V Cypressu příkazy vracejí objekt Chainable, který umožňuje řetězení více příkazů.

Chainable:

- Označuje, že příkaz vrátí chainable objekt, ale nevrací konkrétní prvek.
- Často používáno pro příkazy, které provádí akci (např. kliknutí), ale nevracejí DOM prvek.

Příklad:

```

cy.get('[data-cy="login-button"]').click();
// click() vrátí Chainable<void>, protože nevrací hodnotu.

```

Chainable:

- Označuje, že příkaz vrátí chainable objekt, který vrátí DOM prvek (nebo prvky).

- Vhodné, pokud potřebujete na vybraný prvek provést další akce nebo ověření.

Příklad:

```
cy.get('[data-cy="login-username-input"]').should('be.visible');  
// get() vrací Chainable<Element>, protože vrací DOM prvek.
```

Proč je to důležité:

- Pochopení těchto návratových typů vám pomůže vědět, co očekávat při řetězení příkazů a usnadňuje psaní typově bezpečných testů s IntelliSense.

6. Výhody a nevýhody použití TypeScriptu s Cypress

Výhody:

- **Včasná detekce chyb:**

Kontrola typů zachytí chyby během vývoje dříve, než jsou testy spuštěny.

- **Lepší vývojářský zážitek:**

Bohatý IntelliSense, automatické doplňování a inline dokumentace zjednodušují psaní testů.

- **Lepší údržba:**

Explicitní typy a rozhraní zjednodušují pochopení a refaktorování kódu.

- **Škálovatelnost:**

Jak testovací sada roste, typová bezpečnost pomáhá zvládat komplexnost a předcházet chybám.

Nevýhody:

- **Počáteční režie při nastavení:**

Nastavení TypeScriptu ve stávajícím projektu vyžaduje čas a konfiguraci.

- **Křivka učení:**

Vývojáři noví v TS se musí naučit jeho koncepty a syntaxi.

- **Kompilační krok navíc:**

TypeScript přidává další krok kompilace, což může mírně zpomalit vývojový cyklus oproti čistému JavaScriptu.

- **Riziko překomplikování:**

U velmi malých projektů nemusí být přidaná složitost TypeScriptu opodstatněná.

E. Aktivita

1. Nastavte TypeScript ve svém Cypress projektu:

- Inicializujte Vue nebo Vite projekt (pokud ještě není nastaveno).
- Nainstalujte TypeScript a potřebné typy.

- Vytvořte a nakonfigurujte `tsconfig.json` tak, aby zahrnoval Cypress soubory.
- Převěďte existující JS test na TypeScript.

2. Napište a spusťte jednoduchý Cypress test v TypeScriptu:

- Vytvořte testovací soubor (např. `cypress/e2e/login.spec.ts`), který využívá typové anotace a rozhraní.
- Použijte vlastní příkazy napsané v TypeScriptu k provedení přihlášení.
- Spusťte test a sledujte IntelliSense a kontrolu typů v akci.

3. Refaktorujte stávající testy:

- Najděte části svého testovacího kódu, kde můžete přidat typové anotace pro větší přehlednost.
- Nahraďte jakékoli použití typu `any` správnými rozhraními nebo typy.

Potenciální otázky studentů a odpovědi

1. Q: Co je TypeScript a proč je výhodný pro Cypress testy?

A: TypeScript je staticky typovaný nadmnožina JavaScriptu, která pomáhá odchytit chyby při kompilaci. Zvyšuje kvalitu kódu, poskytuje lepší IntelliSense a doplňování, zlepšuje údržbu a škálovatelnost testů díky zajištění typové bezpečnosti.

2. Q: Musím převést všechny své Cypress testy do TypeScriptu?

A: Ne, ale použití TypeScriptu může být velice přínosné, zejména když vaše testovací sada roste. Můžete začít s několika klíčovými testy a postupně převádět další, jakmile se s TS seznámíte.

3. Q: Jak fungují environmentální proměnné v TypeScriptu s Cypress?

A: Environmentální proměnné se načítají přes `import.meta.env` (ve Vite). TypeScript je považuje za řetězce, pokud neposkytnete vlastní definici typů; proto je důležité převádět je na správné typy (např. pomocí `Number()` pro číselné hodnoty).

4. Q: Jaké jsou hlavní výhody použití vlastních příkazů v TypeScriptu?

A: Vlastní příkazy vám umožňují zabalit opakující se akce, snížit duplikaci kódu a vytvářet vyšší abstrakci. Když jsou napsány v TypeScriptu, poskytují typovou bezpečnost, což usnadňuje údržbu a ladění testů.

5. Q: Jak organizuji své TypeScript soubory pro Cypress testování?

A: Běžná struktura je ukládat testovací soubory do `cypress/e2e` (s příponou `.ts`) a vlastní příkazy, typy a page objecty do `cypress/support`. Dobře organizovaná složková struktura napomáhá udržení přehledu a škálovatelnosti.

6. Q: Co znamená, že TypeScript je „nadmnožina“ JavaScriptu?

A: Znamená to, že veškerý platný JavaScript je také platný TypeScript. TypeScript přidává navíc

funkce jako statické typy a rozhraní, které zvyšují kvalitu kódu, aniž by rušily kompatibilitu s existujícím JavaScriptem.

7. Q: Co je kompilátor a jak funguje s TypeScriptem?

A: Kompilátor je nástroj, který překládá kód z jednoho jazyka do jiného. TypeScript kompilátor (`tsc`) převádí TypeScript na běžný JavaScript, kontroluje typy a zajišťuje, že výstup je kompatibilní s prohlížeči nebo Node.js.

8. Q: Jaký je rozdíl mezi typy a rozhraními v TypeScriptu?

A: Obojí definuje tvar dat. Rozhraní se používají především pro objekty a podporují sloučení deklarací, kdežto typové aliasy jsou flexibilnější a mohou definovat unie, průniky a primitivní typy. Rozhraní používejte pro objekty, typy pro složitější skládání.

9. Q: Jak fungují enumy v TypeScriptu a proč je používat v Cypress testech?

A: Enumy vám umožní definovat sadu pojmenovaných konstant, čímž zlepšují čitelnost a údržbu nahrazením literálů popisnými názvy. V Cypress testech je můžete použít např. pro stavové kódy, role uživatelů nebo jinou sadu předdefinovaných hodnot.

10. Q: Co jsou .d.ts soubory a proč jsou důležité?

A: Soubory deklarací (s příponou .d.ts) poskytují typové informace o knihovnách JS, které nemají vestavěné TypeScript definice. Umožňují TypeScript kompilátoru porozumět externím modulům a zajišťují správnou typovou kontrolu a IntelliSense.

11. Q: Jaké jsou výhody a nevýhody použití TypeScriptu s Cypress?

A: Mezi výhody patří včasné odhalení chyb, lepší zážitek pro vývojáře a jednodušší údržba díky typové bezpečnosti. Nevýhody zahrnují počáteční úsilí při nastavení, potřebu naučit se nové koncepty a krok navíc v podobě kompilace.

12. Q: Co jsou návratové typy v TypeScriptu a proč bych je měl používat?

A: Návratové typy explicitně určují typ hodnoty, kterou funkce vrací, což pomáhá zachytit chyby a zlepšuje přehlednost během kompilace.

13. Q: Co znamená Chainable vs. Chainable v Cypress?

A: `Chainable<void>` označuje, že Cypress příkaz vrací chainable objekt bez DOM prvku (např. klikací akce), zatímco `Chainable<Element>` znamená, že příkaz vrací DOM prvek, na kterém můžete dále provádět akce nebo ověřování.

Zdroje

- **TypeScript s Cypress dokumentace:**
[Cypress TypeScript Support](#)
- **Oficiální dokumentace TypeScriptu:**
[TypeScript Handbook](#)

- **Příklady Cypress testů:**

Najděte open-source projekty na GitHubu, které ukazují použití TypeScriptu s Cypress.